

1 Theory

A shift register is a sequential logic circuit used for data storage and transfer. It operates by shifting data bits in a specified direction (left or right) with each clock pulse. Shift registers are classified into four main types based on their data input and output configurations: Parallel-In Parallel-Out (PIPO), Serial-In Parallel-Out (SIPO), Parallel-In Serial-Out (PISO), and Serial-In Serial-Out (SISO).

PIPO

In a PIPO shift register, data is loaded into the register in parallel (all bits simultaneously) and output in parallel. Thus, the total number of clock cycles required is 1 (1 for loading + 0 for reading). It is primarily used when high-speed data processing is required, as the parallel nature of both input and output minimizes delays.

SIPO

A SIPO register accepts data serially, one bit at a time, and outputs all the data bits simultaneously in parallel. Thus, the total number of clock cycles required is n (n for loading + 0 for reading). It is commonly used for converting serial data streams into parallel formats, such as in communication systems.

PISO

In a PISO register, data is loaded into the register in parallel and shifted out serially, one bit at a time. Thus, the total number of clock cycles required is n (1 for loading + $(n-1)$ for reading). This type is ideal for applications requiring parallel-to-serial data conversion, such as transmitting data over serial communication lines.

SISO

The SISO shift register handles data both input and output serially, one bit at a time. Thus, the total number of clock cycles required is $2n-1$ (n for loading + $(n-1)$ for reading). It is the simplest type and is widely used for basic data storage or shifting operations within a system.

2 Implementation and Observation

2.1 PIPO

VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```

entity pipo is
    port( clk: in std_logic;
          reset: in std_logic;
          d: in std_logic_vector(3 downto 0);
          q: out std_logic_vector(3 downto 0)
        );
end pipo;

```

architecture Behavioral of pipo is

begin

```

process(clk, reset, d)
begin
    if(reset = '1') then
        q <= "0000";
    elsif(rising_edge(clk)) then
        q(3 downto 0) <= d(3 downto 0);
    end if;
end process;
end Behavioral;

```

Test Bench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity pipo_tb is
    --Port ( );
end pipo_tb;

```

architecture Behavioral of pipo_tb is

```

component pipo is
    port( clk: in std_logic;
          reset: in std_logic;
          d: in std_logic_vector(3 downto 0);
          q: out std_logic_vector(3 downto 0)
        );
end component;

```

```

signal clk: std_logic := '0';
signal reset: std_logic;
signal d: std_logic_vector(3 downto 0);
signal q: std_logic_vector(3 downto 0);

```

```

begin
DUT: pipo port map(
clk => clk,
reset => reset,
d => d,
q => q
);

clk_process: process
begin
clk <= not clk;
wait for 5 ns;
end process;

process
begin
reset <= '0';
d <= "1000";
wait for 10 ns;

reset <= '0';
d <= "0001";
wait for 10 ns;

reset <= '1';
d <= "1111";
wait;
end process;

end Behavioral;

```

RTL Schematic Diagram

RTL design is used to describe the behavior of the PIPO as shown in figure 1

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the PIPO circuit as shown in figure 2

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of PIPO as shown in figure 3.

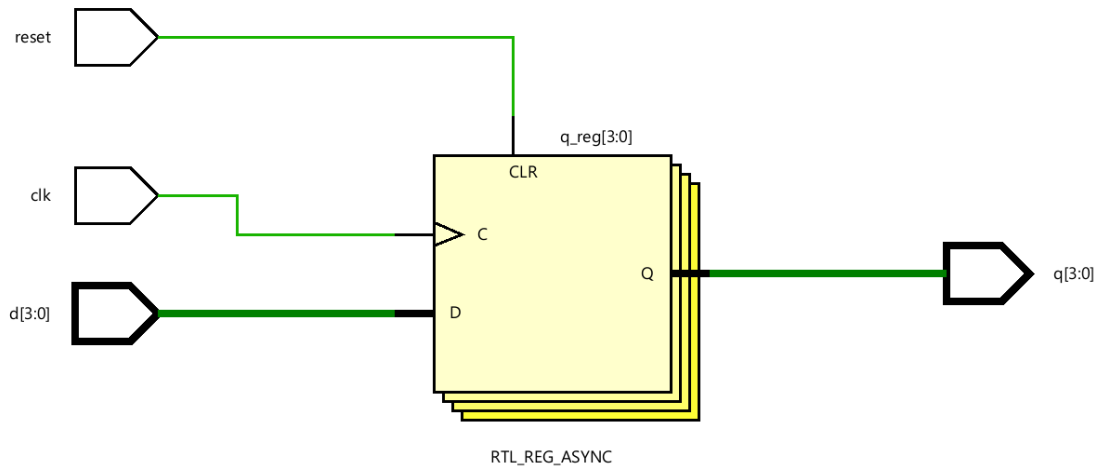


Figure 1: RTL Schematic Diagram of PIPO

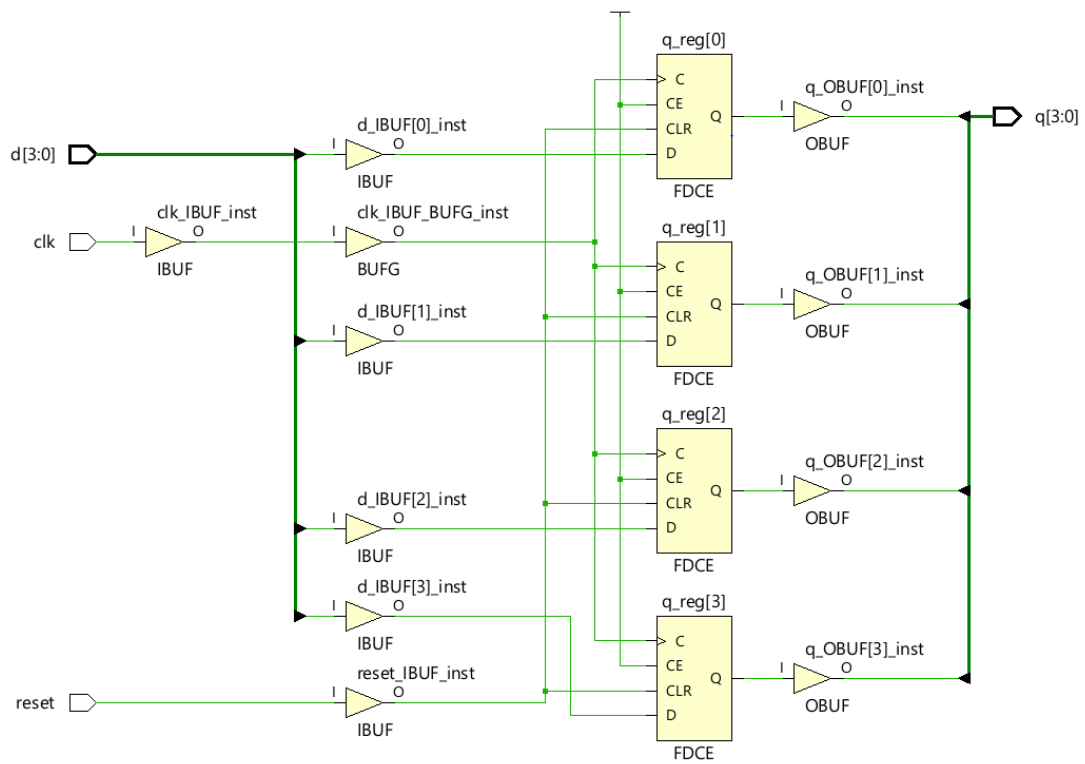


Figure 2: Synthesis Schematic Diagram of PIPO

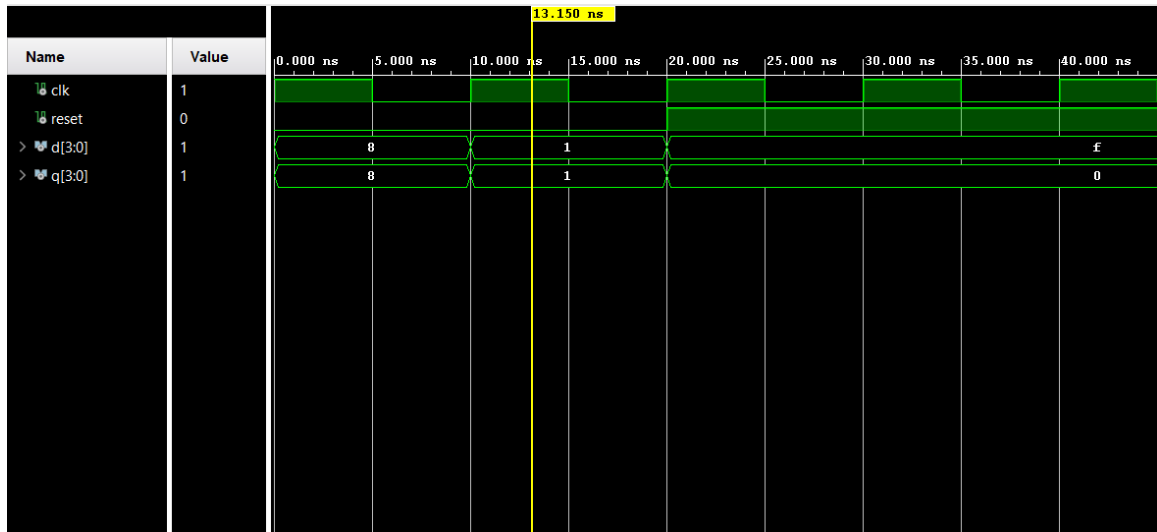


Figure 3: Timing Diagram of PIPO

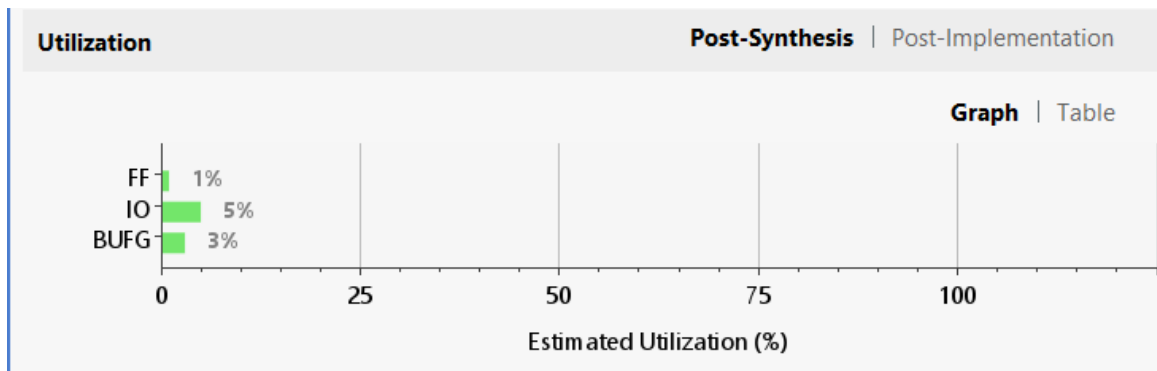


Figure 4: Device Utilization of PIPO

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 4.

Power Estimation

Figure 5 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a PIPO circuit.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **1.079 W**
Design Power Budget: **Not Specified**
Process: **typical**
Power Budget Margin: **N/A**
Junction Temperature: **30.2°C**
Thermal Margin: 54.8°C (11.4 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 4.8°C/W
Power supplied to off-chip devices: 0 W
Confidence level: **Low**
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

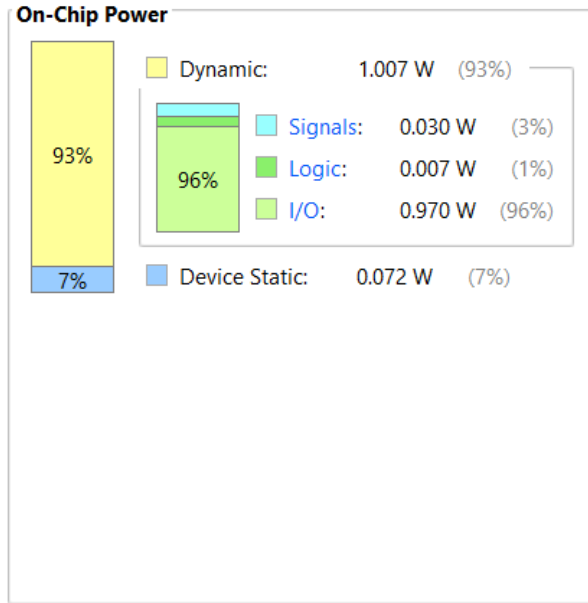


Figure 5: Power Estimation of PIPO

2.2 SIPO

VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sipo is
    port( clk: in std_logic;
          reset: in std_logic;
          d: in std_logic;
          q: out std_logic_vector(3 downto 0)
        );
end sipo;

architecture Behavioral of sipo is
    signal temp: std_logic_vector(3 downto 0);
begin

    process(clk, reset, d)
    begin
        if(reset = '1') then
            q <= "0000";
        elsif(rising_edge(clk)) then
            temp(3) <= d;
```

```
temp(2 downto 0) <= temp(3 downto 1);
end if;
```

```
q(3 downto 0) <= temp(3 downto 0);
end process;
end Behavioral;
```

Test Bench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity sipo_tb is
    --Port ( );
end sipo_tb;
```

```
architecture Behavioral of sipo_tb is
    component sipo is
        port( clk: in std_logic;
              reset: in std_logic;
              d: in std_logic;
              q: out std_logic_vector(3 downto 0)
            );
    end component;
```

```
    signal clk: std_logic := '0';
    signal reset: std_logic;
    signal d: std_logic;
    signal q: std_logic_vector(3 downto 0);
```

```
begin
    DUT: sipo port map(
        clk => clk,
        reset => reset,
        d => d,
        q => q
    );
```

```
    clk_process: process
    begin
        clk <= not clk;
        wait for 5 ns;
    end process;
```

```
process
```

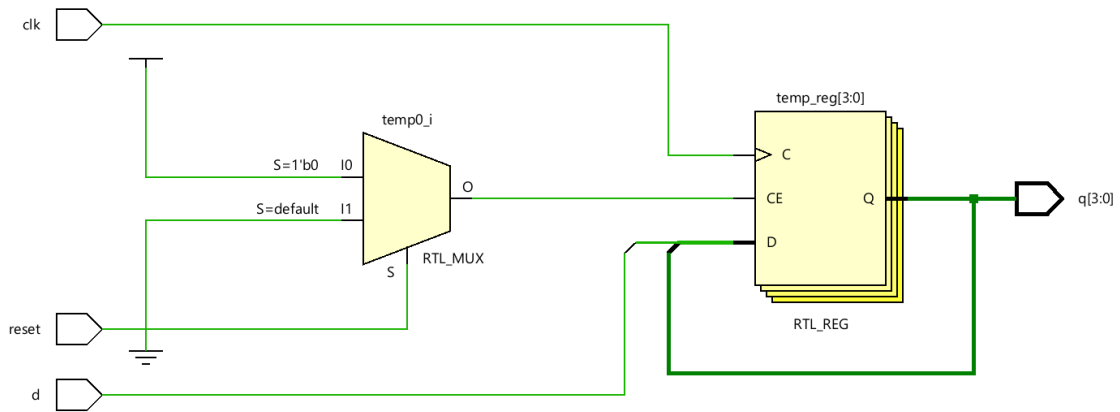


Figure 6: RTL Schematic Diagram of SIPO

```

begin
reset <= '0';
d <= '1';
wait for 10 ns;

reset <= '0';
d <= '0';
wait for 10 ns;

reset <= '0';
d <= '0';
wait for 10 ns;

reset <= '0';
d <= '1';
wait for 10 ns;

reset <= '1';
d <= '1';
wait for 10 ns;

wait;
end process;

end Behavioral;

```

RTL Schematic Diagram

RTL design is used to describe the behavior of the SIPO as shown in figure 6

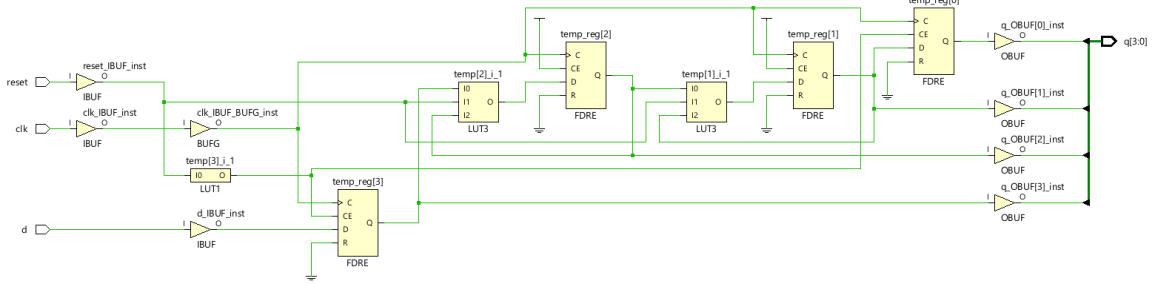


Figure 7: Synthesis Schematic Diagram of SIPO

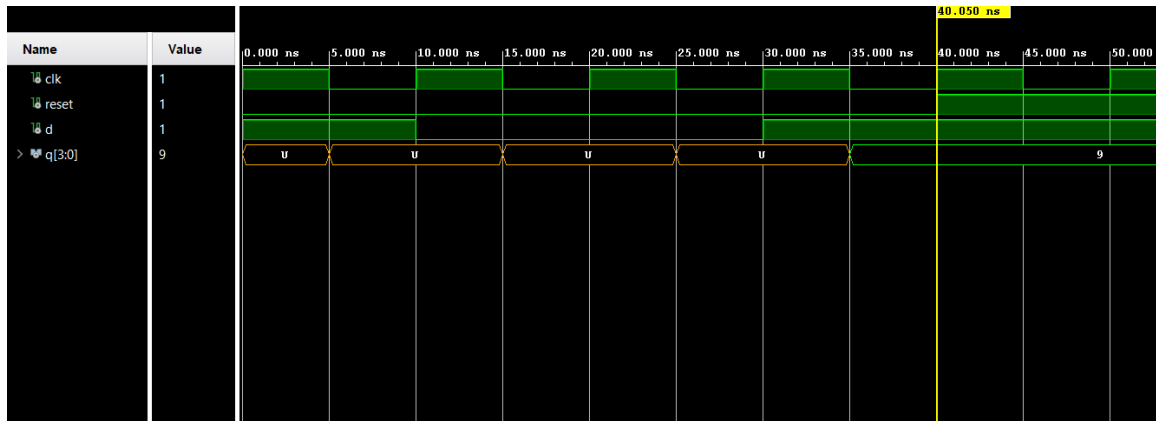


Figure 8: Timing Diagram of SIPO

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the SIPO circuit as shown in figure 7

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of SIPO as shown in figure 8.

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 9.

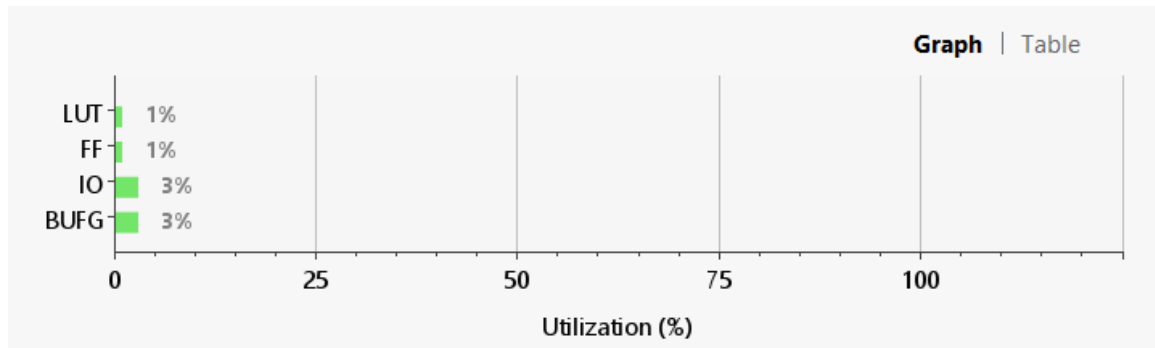


Figure 9: Device Utilization of SIPO

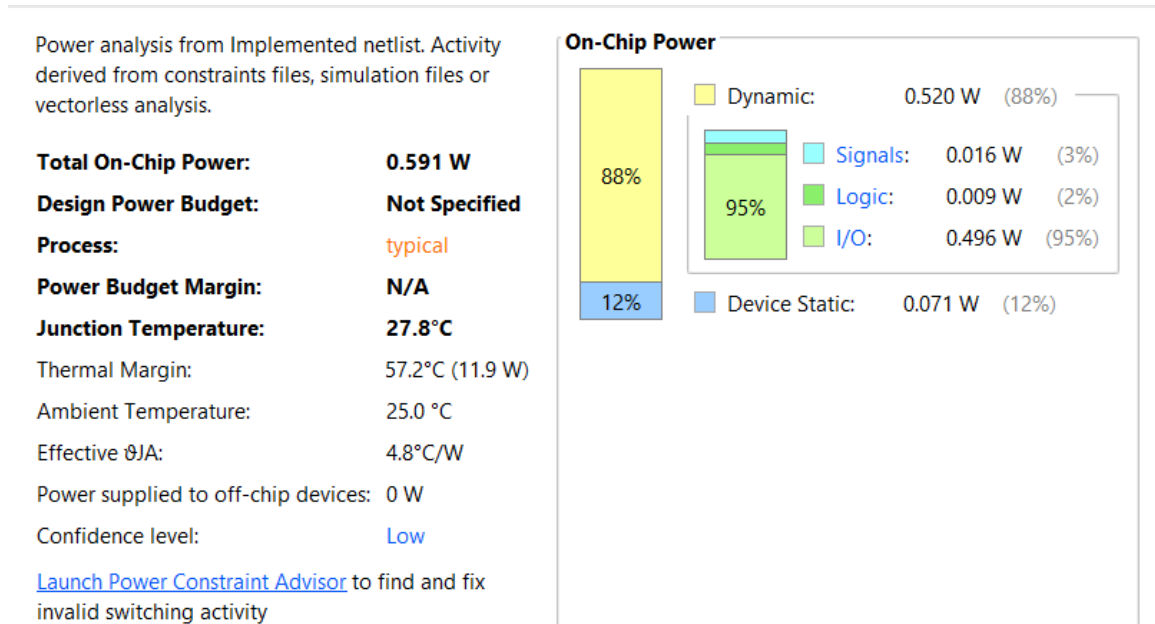


Figure 10: Power Estimation of SIPO

Power Estimation

Figure 10 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a SIPO circuit.

2.3 PISO

VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity PISO is
```

```

    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           load: in STD_LOGIC;
           d : in STD_LOGIC_VECTOR (3 downto 0);
           q : out STD_LOGIC);
end PISO;

architecture Behavioral of PISO is
    signal temp:std_logic_vector(3 downto 0):=(others=>'0');
begin
    process(clk,reset,d)
    begin
        if(reset='1') then
            temp<= (others=>'0');
        elsif (rising_edge(clk)) then
            if load='1' then
                temp<=d;
            else
                temp <= temp( 2 downto 0) & '0';
            end if;
        end if;
    end process;
    q<= temp(3);
end Behavioral;

```

Test Bench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity PISO_tb is
end PISO_tb;

architecture Behavioral of PISO_tb is
    component PISO
        port( clk : in STD_LOGIC;
              reset : in STD_LOGIC;
              load: in STD_LOGIC;
              d : in STD_LOGIC_VECTOR(3 downto 0);
              q : out STD_LOGIC);
    end component;

    signal clk : STD_LOGIC:='0';
    signal reset : STD_LOGIC;
    signal load : STD_LOGIC;
    signal d :STD_LOGIC_VECTOR(3 downto 0) :="0000";

```

```

signal q : STD_LOGIC;

begin
 uut: PISO port map(
    clk=>clk,
    reset=>reset,
    load=>load,
    d=>d,
    q=>q);
 stimulus:process
 begin
    clk<= not clk;
    wait for 10 ns;
 end process;
 process
 begin
    reset<='1';
    wait for 20 ns;
    reset<='0';
    wait for 20 ns;

    load<='1';
    d<="1001";
    wait for 10 ns;
    load<='0';
    wait for 10 ns;
 wait;
 end process;
end Behavioral;

```

RTL Schematic Diagram

RTL design is used to describe the behavior of the PISO as shown in figure 11

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the PISO circuit as shown in figure 12

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of PISO as shown in figure 13.

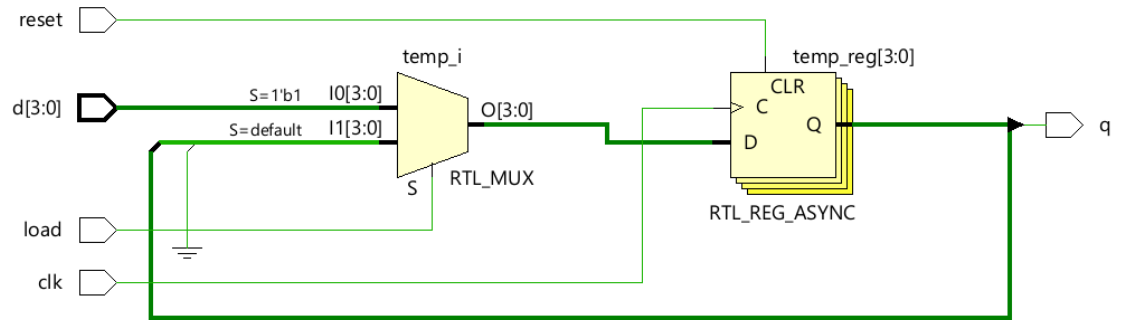


Figure 11: RTL Schematic Diagram of PISO

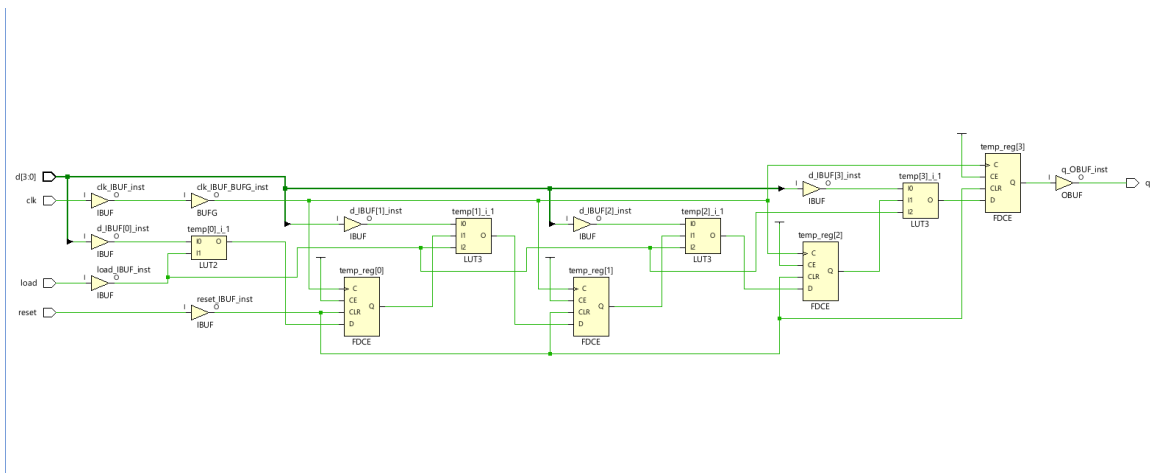


Figure 12: Synthesis Schematic Diagram of PISO

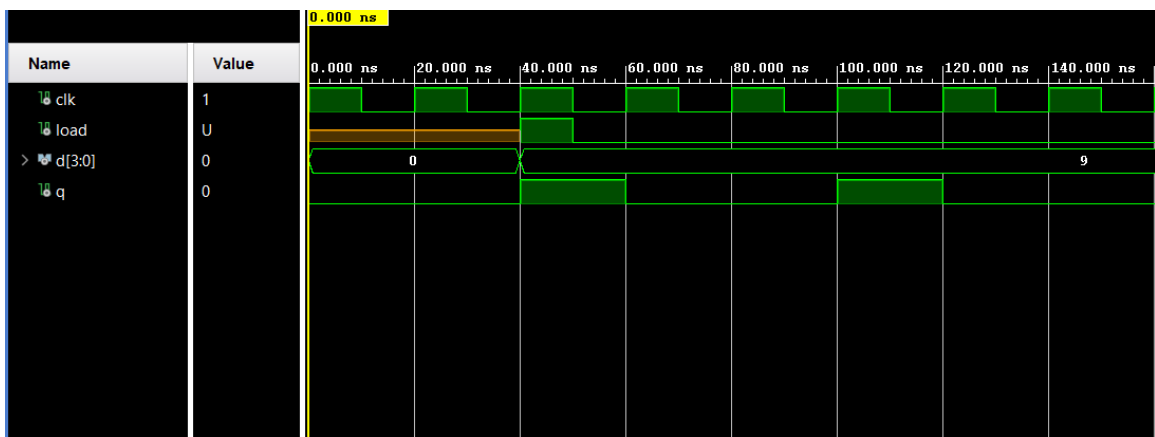


Figure 13: Timing Diagram of PISO

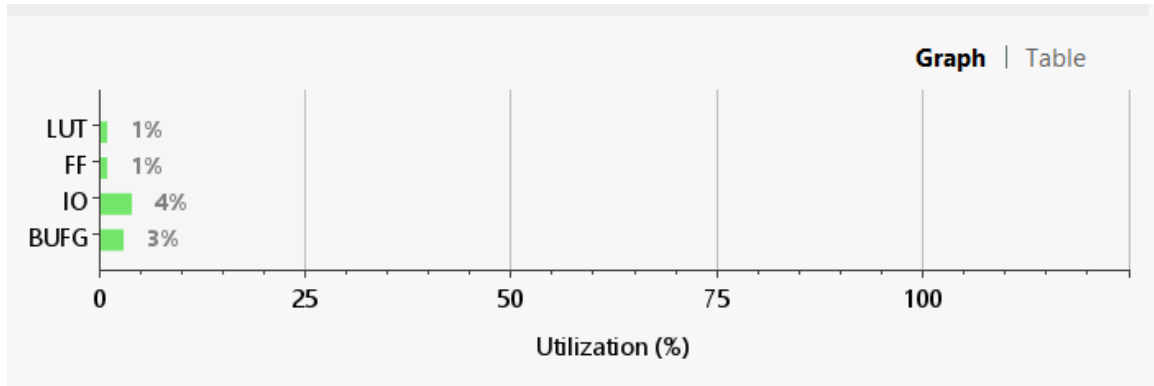


Figure 14: Device Utilization of PISO

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables (LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 14.

Power Estimation

Figure 15 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a PISO circuit.

2.4 SISO

VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity siso is
    port( clk: in std_logic;
          reset: in std_logic;
          d: in std_logic;
          q: out std_logic
        );
end siso;

architecture Behavioral of siso is
    signal temp: std_logic_vector(3 downto 0);
begin
```

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **0.523 W**
Design Power Budget: **Not Specified**
Process: **typical**
Power Budget Margin: **N/A**
Junction Temperature: **27.5°C**
Thermal Margin: 57.5°C (12.0 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 4.8°C/W
Power supplied to off-chip devices: 0 W
Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

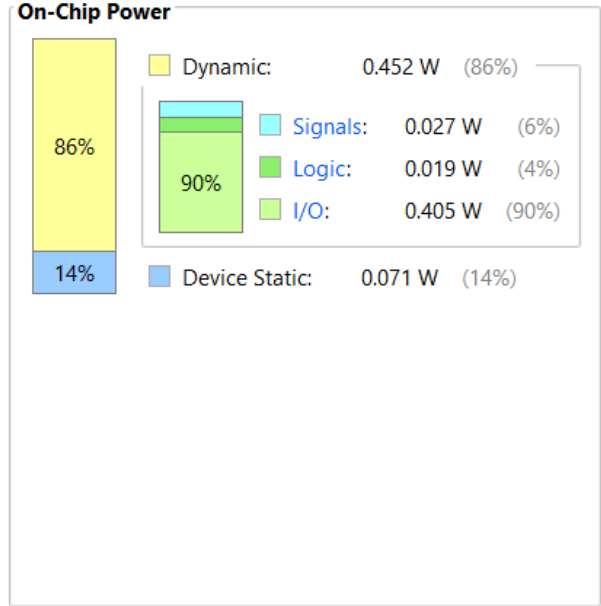


Figure 15: Power Estimation of PISO

```
process(clk, reset, d)
begin
if(reset = '1') then
q <= '0';
elsif(rising_edge(clk)) then
temp(3) <= d;
temp(2 downto 0) <= temp(3 downto 1);
q <= temp(0);
end if;
end process;
end Behavioral;
```

Test Bench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity siso_tb is
    --Port ( );
end siso_tb;

architecture Behavioral of siso_tb is
    component siso is
        port( clk: in std_logic;
```

```

        reset: in std_logic;
        d: in std_logic;
        q: out std_logic
    );
end component;

signal clk: std_logic := '0';
signal reset: std_logic;
signal d: std_logic;
signal q: std_logic := '0' ;

begin
DUT: siso port map(
    clk => clk,
    reset => reset,
    d => d,
    q => q
);

clk_process: process
begin
    clk <= not clk;
    wait for 5 ns;
end process;

process
begin
    reset <= '0';
    d <= '1';
    wait for 10 ns;

    reset <= '0';
    d <= '0';
    wait for 10 ns;

    reset <= '0';
    d <= '0';
    wait for 10 ns;

    reset <= '0';
    d <= '1';
    wait for 10 ns;

    reset <= '0';
    wait for 30 ns;

```

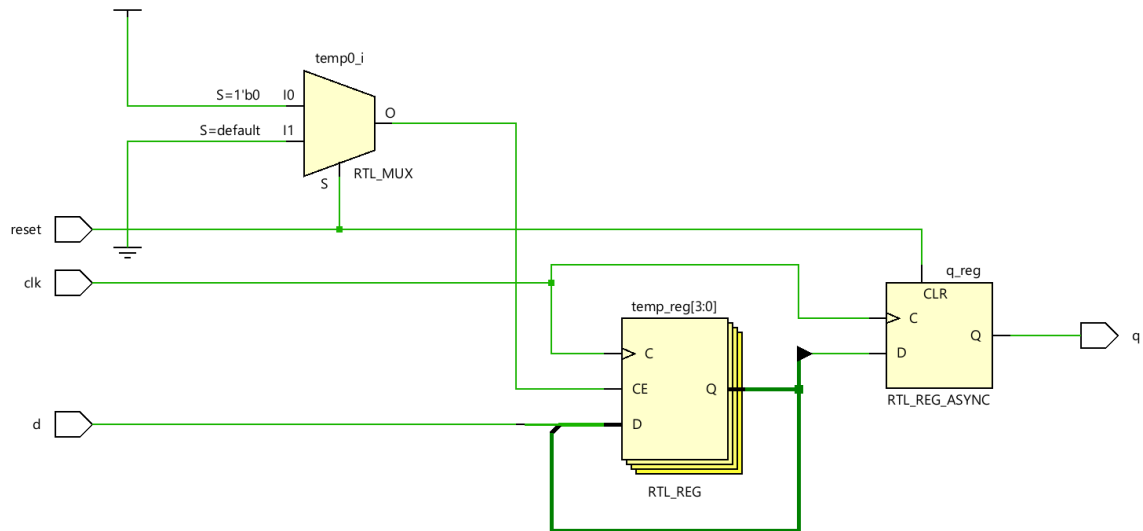


Figure 16: RTL Schematic Diagram of SISO

```
wait;
end process;

end Behavioral;
```

RTL Schematic Diagram

RTL design is used to describe the behavior of the SISO as shown in figure 16

Synthesis Schematic Diagram

Schematic design is used to describe the implementation of the SISO circuit as shown in figure 17

Timing diagram

Timing diagram test is done to check whether the VHDL program and schematic diagram is valid for the given truth table of SISO as shown in figure 18.

Device Utilization

IO (Input/Output) refers to the physical FPGA pins used for communication with external devices, tracking how many of these pins are utilized for inputs and outputs. Utilization, on the other hand, measures the internal FPGA resource consumption, such as lookup tables

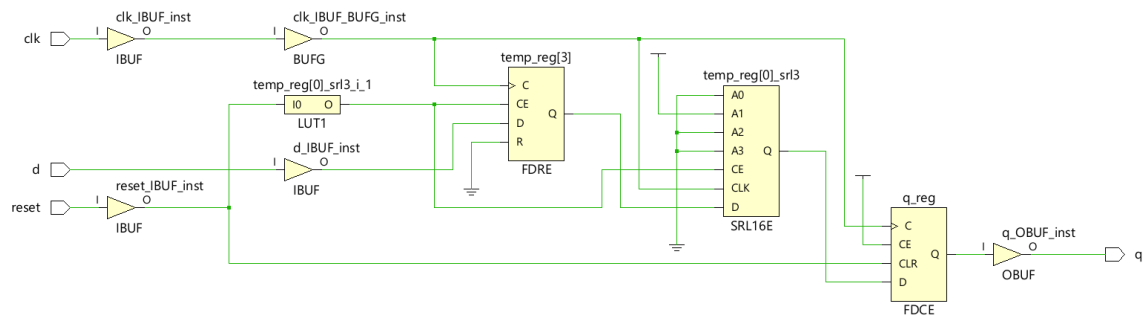


Figure 17: Synthesis Schematic Diagram of SISO

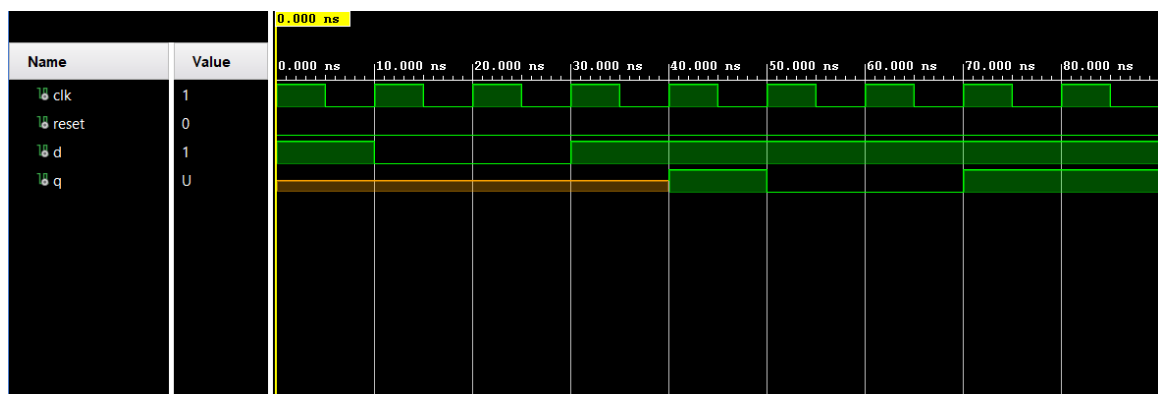


Figure 18: Timing Diagram of SISO

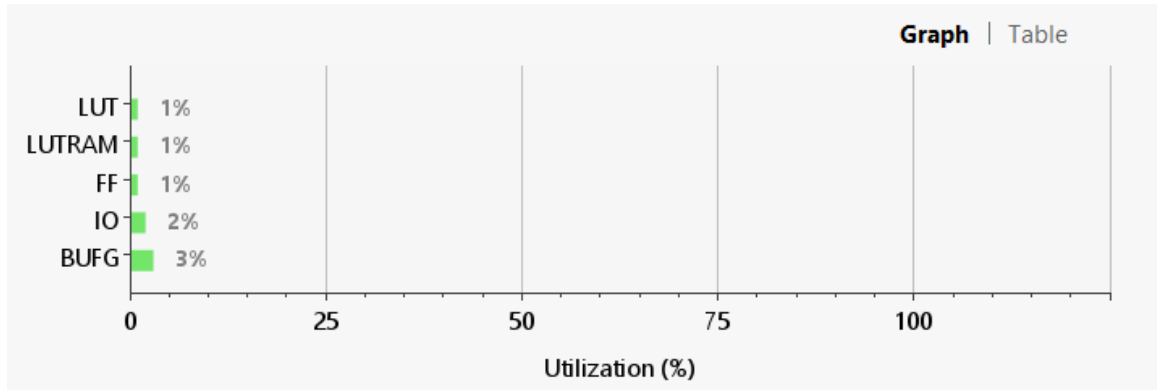


Figure 19: Device Utilization of SISO

(LUTs), flip-flops, block RAM (BRAM), and DSP slices. Together, these concepts provide a comprehensive view of FPGA design efficiency, as shown in figure 19.

Power Estimation

Figure 15 shows estimation of static power (no change in input and output), dynamic power (power required when VHDL program is burned in FPGA) and other related power data of a SISO circuit.

3 Conclusion

In this lab, we implemented the VHDL code for PIPO, SIPO, PISO and SISO on the Atrix-7 FPGA (part: xc7a50tcsg324-1, package: csg324) using Vivado software. Each design was synthesized, simulated, and implemented, which confirmed their correct functionality. The process began with simulation to verify the timing diagrams, followed by linting to check for syntax errors. Afterward, the designs were synthesized into a hardware logic circuit, and during the implementation phase, power and timing characteristics were analyzed, displayed and studied.

Each flip-flop was implemented and verified through simulation, confirming its expected behavior and showcasing its role in sequential logic systems. This lab provided practical experience in using Vivado for digital design and strengthened the understanding of shift registers as essential components in modern digital applications.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **0.333 W**
Design Power Budget: **Not Specified**
Process: **typical**
Power Budget Margin: **N/A**
Junction Temperature: **26.6°C**
 Thermal Margin: 58.4°C (12.2 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 4.8°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

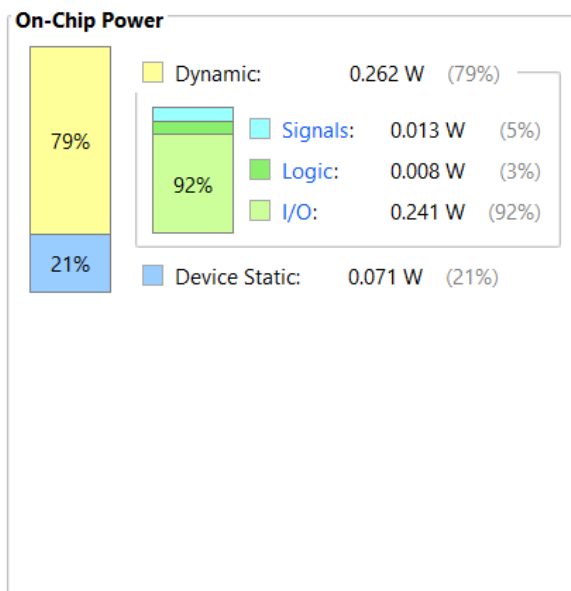


Figure 20: Power Estimation of SISO